HLD & Architecture – Option B (Next.js + Supabase)

Oct 26, 2026 · @Krish G

1. Overview and principles

One multi-tenant Next.js web app, hosted on AWS Amplify, with Supabase (Postgres, Auth, Storage) as the backend; Bindushree Biomedicals is tenant #1 and every later business is another tenant on the same app.

This document covers the BRD's Phase 1-3 scope plus the planned Tally and payments phase. It is written for one or two developers building the first version.

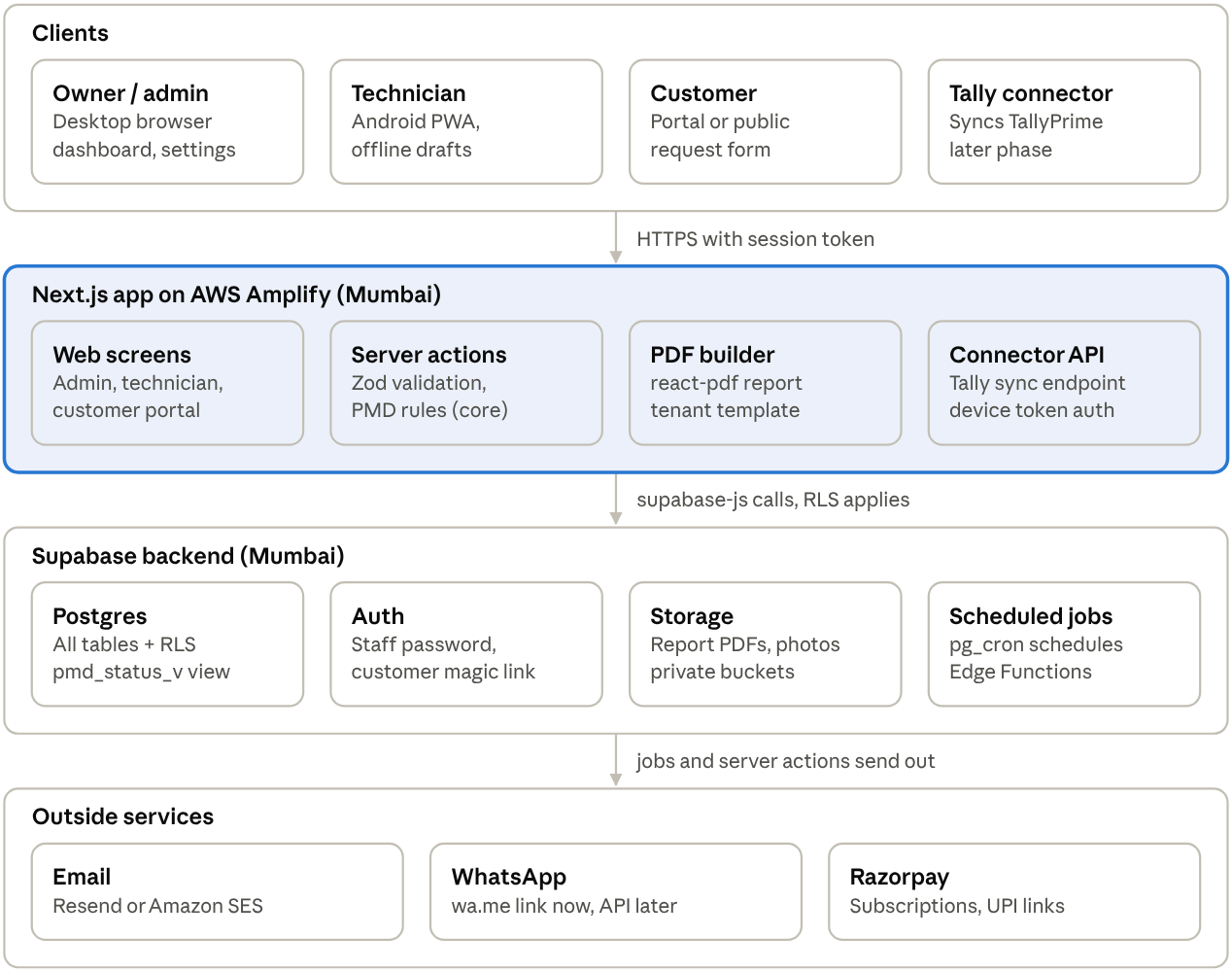
Architecture principles

1. **Multi-tenant from day one.** Every business row carries a `tenant_id`; Postgres row-level security (RLS) enforces isolation in the database, not only in code.
2. **One codebase, three surfaces.** The same web app serves admin desktop, technician mobile (installable PWA), and the customer portal.
3. **Compute, don't store, status.** PMD colours are derived from due dates at query time, so they are never stale.
4. **Tally owns money.** The app stores service data; invoices and balances are read from Tally, never edited twice.
5. **Managed services over servers.** No VMs to patch; scale by upgrading plans.
6. **Portable.** Plain Postgres and open-source Supabase, so the app can move to AWS RDS or a customer's own server later.
7. **Data in India.** Database and files in the Mumbai region for latency and the Digital Personal Data Protection Act, 2023.

2. Component architecture

Four layers: clients call only the Next.js app; the app talks to Supabase with the user's own session, so the database applies tenant rules on every query.

Every request goes through the Next.js app; Supabase enforces tenant rules



component architecture · 4 layers

Component	Responsibility
Web screens	Server-rendered pages for each role; installable as a PWA on phones
Server actions	All writes go here: validate with Zod, apply PMD rules from `packages/core`, write to Postgres
PDF builder	Turns a finalised report (data + tenant template) into a PDF and saves it to Storage
Connector API	Receives Tally sync data and hands out queued invoices, authenticated by a per-device token

Component	Responsibility
Postgres	System of record for service data, with RLS and audit triggers
Auth	Logins, sessions, and tenant / role claims in the token
Storage	PDFs, photos and logos in private, per-tenant folders
Scheduled jobs	Reminders, digests, retries and subscription checks on a timer

3. Technology stack

Everything is TypeScript end to end, so one developer can work across the web app, server code and the Tally connector.

Layer	Choice	Why
Web framework	Next.js (App Router, React, TypeScript)	One codebase for admin, technician and customer screens; server actions for writes
UI	Tailwind CSS + shadcn/ui components	Fast, consistent, mobile-friendly forms and tables
Hosting	AWS Amplify Hosting (Mumbai region)	Already used for the Bindushree website; deploys from GitHub on each push
Database	Supabase Postgres (Mumbai region)	Relational data, SQL reporting, row-level security
Auth	Supabase Auth	Email + password for staff; phone OTP or magic link for customers
File storage	Supabase Storage	Service report PDFs, photos, logos; private buckets per tenant
Background jobs	Supabase pg_cron + Edge Functions	Daily reminders, digests, retries without a separate server

Layer	Choice	Why
PDF generation	@react-pdf/renderer (server side)	No headless browser needed, works on serverless hosting
Validation	Zod	One schema validates forms and API input
Email	Resend or Amazon SES	Transactional email with PDF attachments
WhatsApp	Share link (wa.me) now; Business API via an Indian provider later	Free at launch; automation when volume justifies it
Payments (later)	Razorpay	SaaS subscriptions and UPI payment links in India
Offline	PWA service worker + IndexedDB	Technician drafts survive weak signal
Tally connector (later)	Tauri (Rust shell + TypeScript UI)	Small Windows installer, low memory
Monitoring	Sentry + Supabase logs	Errors with user and tenant context
Testing	Vitest (unit), Playwright (end to end)	Covers PMD date rules and the report flow

4. Multi-tenancy, roles and security

All businesses share one database, and Postgres row-level security guarantees each user sees only their own business's rows, and customers only their own records.

Tenant isolation

- Every business table has a `tenant_id` column, indexed, and a foreign key to `tenants`.
- On login, the user's `tenant_id` and `role` are added to their session token (JWT custom claims).
- RLS policy on every table: rows are visible only where `tenant_id` matches the token.
- Customer users get a second filter: only rows linked to their own `customer_id`.
- The service-role key (which bypasses RLS) is used only in server-side jobs, never sent to the browser.

Roles

Role	Scope	Typical user
platform_admin	All tenants (support and billing only)	You
owner	One tenant, all data and settings	Muttu
technician	One tenant; customers, units, visits, reports	Field engineers
customer	Own equipment, reports and requests only	Lab or hospital contact

Authentication

- Staff: email + password with optional two-step verification.
- Customers: magic link by email at launch. Phone OTP needs an SMS provider and Indian DLT template registration, so it comes later (or OTP over WhatsApp).
- Phase 1 service requests can come through a public form per tenant with a CAPTCHA, no login needed.

Data protection

- Encryption in transit (HTTPS) and at rest (Supabase default).
- Private storage buckets; PDFs shared through signed links that expire (for example 7 days).
- Audit log table records who created, edited, finalised or sent each record.
- Data hosted in India (Mumbai); export and deletion on request, in line with the DPDP Act, 2023.
- Rate limits on login, public forms and the connector API.

5. Data model

Fifteen Postgres tables cover Phases 1-3, with the installed unit at the centre; every table below also has `id` (UUID), `tenant_id`, `created_at`, `updated_at` and `created_by`.

Table	Key columns	Notes
tenants	name, gstin, logo_path, plan, settings (JSON: reminder_days, default_pmd_months, report_template)	One row per business
profiles	user_id, role, full_name, mobile, customer_id (customers only)	Links Supabase login to tenant and role
customers	name, contact_person, mobile, whatsapp, email, address, city, pin, gstin, tally_ledger_name	Lab or hospital
equipment_types	name, category	Tenant's own master list
units	customer_id, equipment_type_id, make, model, serial_no, installed_on, pmd_interval_months (3 or 6), pmd_start_date, status	One installed machine
pmd_schedule	unit_id, due_date, completed_on, completed_event_id, skip_reason	One row per due PMD
service_events	unit_id, type (installation / breakdown / preventive), event_date, technician_id, complaint, work_done, status, billable, amount, invoice_ref, request_id	One visit
chemicals	name, category (reagents, biochemistry, hematology, rapid cards, consumables)	Tenant's master list
service_event_chemicals	event_id, chemical_id, quantity, batch_no, expiry	Many-to-many
service_reports	event_id, report_no, status (draft / final), version, data (JSON), pdf_path, signed_by_customer, finalised_at	PDF snapshot per version

Table	Key columns	Notes
report_sends	report_id, channel (email / whatsapp), recipient, sent_at, status	Delivery log
service_requests	customer_id, unit_id, problem, preferred_date, photo_path, status (open / assigned / closed), assigned_to	From customer or form
notifications	user_id, kind, payload, read_at	In-app reminders
audit_log	table_name, row_id, action, changed_by, diff (JSON)	Written by database triggers
subscriptions	plan, status, razorpay_subscription_id, renews_on	Your SaaS billing per tenant

Derived view `pmd_status_v`: returns each open PMD with status = Completed when `completed_on` is set, Overdue when `due_date` is before today, Due when within the tenant's `reminder_days`, else Scheduled. The dashboard reads this view, so colours are always current without a nightly update.

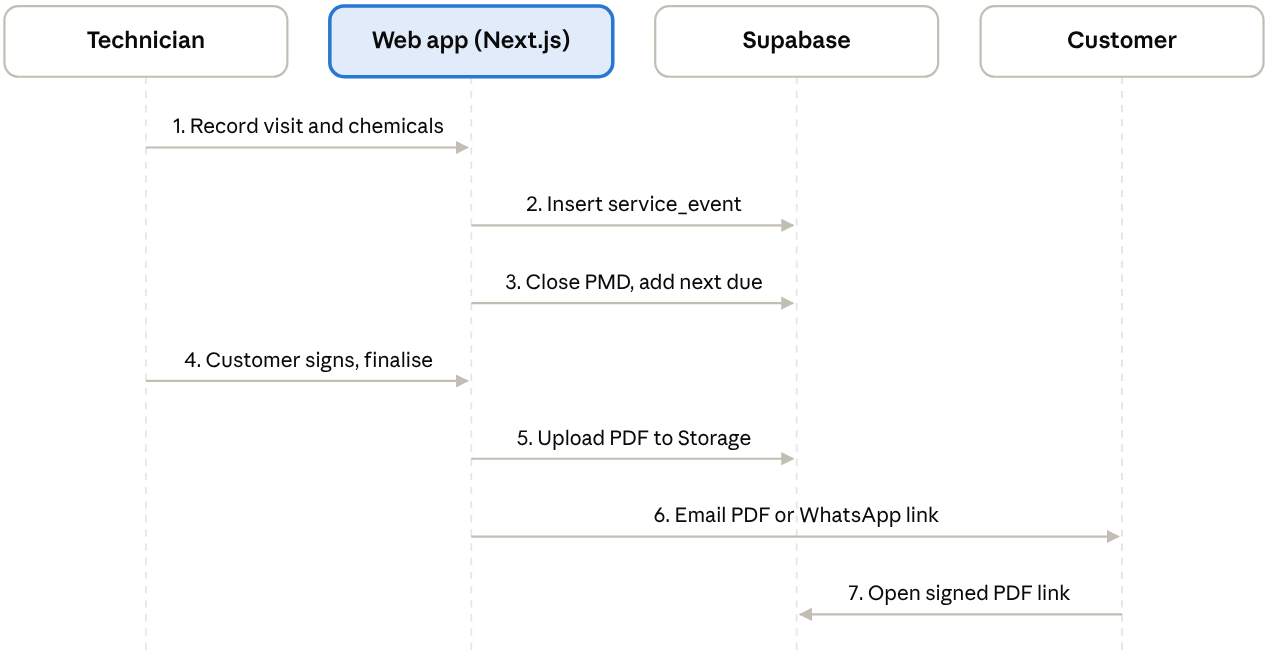
Tally phase adds: `tally_connections` (tenant, device_token_hash, company_name, last_sync_at, status), `receivables` (customer_id, voucher_no, bill_date, due_date, amount, outstanding; refreshed from Tally) and `sync_queue` (operation, payload, idempotency_key, attempts, status).

6. Key flows

The service visit is the core flow: one save records the work, closes the PMD and books the next due date in a single database transaction.

Service visit to report

One finalised visit closes the PMD, books the next and reaches the customer



service visit sequence · 7 steps

Steps 2 and 3 run in one transaction, so a PMD is never marked done without its visit record. Step 4 locks the report version; later edits create version 2 and a new PDF.

Installation on site (new customer)

1. Technician creates the customer, then adds the installed unit with type, serial number and installation date.
2. App defaults PMD interval to 6 months (editable to 3) and the start date to the installation date.
3. App inserts the Installation event and the first `pmd_schedule` row (start date + interval).

Daily PMD reminders

1. At 07:30 IST the `pmd-reminders` job reads `pmd_status_v` per tenant.
2. PMDs that turned Due or Overdue since yesterday create in-app notifications for the owner (and assigned technician).
3. The owner gets one digest email listing overdue (red) then due (yellow) items with customer and unit.
4. Each notification records its date, so a rerun sends nothing new.

Customer service request

1. Customer submits the public form or portal request (unit, problem, preferred date, optional photo).
2. Owner is notified, assigns a technician; status moves to Assigned.
3. The technician's visit is saved as a Breakdown event linked by `request_id`; finalising the report closes the request.

7. Background jobs and integrations

Five scheduled jobs and four outside services run the automation; all jobs are idempotent, so a rerun never sends a reminder twice.

Scheduled jobs (times in IST)

Job	When	What it does
pmd-reminders	Daily 07:30	For each tenant, finds PMDs entering Due or Overdue; creates in-app notifications and emails the owner a digest
report-send-retry	Every 15 minutes	Retries failed email / WhatsApp sends, up to 3 times
storage-cleanup	Weekly	Removes orphaned draft photos older than 30 days
subscription-check	Daily 02:00	Syncs Razorpay subscription status; moves unpaid tenants to read-only after a grace period
tally-health (later)	Hourly	Flags connectors with no sync for more than 2 hours

Integrations

Service	Direction	How
Email (Resend or SES)	Out	Report PDF attached or linked; digest emails; bounces logged to `report_sends`
WhatsApp	Out	Phase 1: `wa.me` link pre-filled with the customer number and a signed PDF link. Later: Business API through an Indian provider using approved utility templates

Service	Direction	How
Razorpay	In and out	Subscription billing for tenants; webhooks update `subscriptions`; later UPI links for end-customer payments
TallyPrime (later)	Both ways	Through the Tally connector, below

Tally connector (later phase)

1. The business owner downloads the Windows connector (Tauri) and pairs it with a one-time code shown in the web app; the app stores only a hash of the device token.
2. The connector calls TallyPrime's XML-over-HTTP server on the same PC (localhost, port configured in Tally, typically 9000).
3. Every 15 minutes it pulls outstanding bills, receipts and ledger list, and sends them over HTTPS to a `/api/tally/sync` endpoint, which refreshes `receivables`.
4. It polls `sync_queue` for invoices to create, posts each to Tally as a sales voucher, and returns the voucher number. Each item carries an idempotency key, so a retry never creates a duplicate.
5. If Tally or the PC is off, items wait in the queue; the web app shows the last successful sync time.

8. Deployment, environments and repository

Three environments, each with its own Amplify branch and Supabase project, promoted by Git merges; database changes ship as versioned SQL migrations.

Environment	Git branch	Amplify	Supabase project	Used for
Local	feature branches	`next dev`	Supabase CLI (local Docker)	Development
Staging	`staging`	staging.app domain	Free tier project	Testing with Muttu
Production	`main`	app domain (e.g. app.bindushreebiomedicals.com, later your SaaS domain)	Pro plan, Mumbai	Live tenants

Release flow

1. Pull request runs type-check, unit tests and migration dry-run in GitHub Actions.
2. Merge to `staging` deploys automatically; Playwright smoke tests run against it.
3. Merge to `main` runs `supabase db push` for migrations, then Amplify deploys the app.

Repository (new monorepo, separate from the marketing website repo)

```
service-app/
  apps/
    web/              Next.js app (admin, technician, customer portal)
    tally-connector/  Tauri desktop app (later phase)
  packages/
    db/               SQL migrations, RLS policies, seed data, generated
types
    core/             shared rules: PMD date maths, status, validation (Zod)
    pdf/              service report templates
  supabase/
    functions/        Edge Functions for scheduled jobs and webhooks
```

Keeping PMD rules in `packages/core` lets the web app, jobs and tests use exactly the same date logic.

9. Non-functional design

The targets from the BRD are met with managed-service defaults plus three additions: offline drafts, point-in-time recovery once there are paying tenants, and error monitoring.

Area	Target	How it is met
Performance	Dashboard and search under 3 s on 4G	Server-rendered pages, indexes on `tenant_id` + `due_date` / `customer_id`, pagination at 50 rows
Offline	Technician can finish a report with no signal	PWA caches the app; report drafts and photos saved to IndexedDB; upload on reconnect with conflict check on `version`
Availability	Business hours, no planned downtime	Managed hosting; migrations written to be backward compatible
Backups	Restore any day in the last 7	Supabase Pro daily backups; add point-in-time recovery when paying tenants exceed about 10
Monitoring	Know about errors before customers call	Sentry for app errors (tagged with tenant and role); alerts on failed jobs and stale Tally syncs
Scale	500 tenants, 20,000 units without redesign	Single Postgres with indexes; upgrade compute size first; read replica only if reports slow down
Security	No cross-tenant access	RLS on every table, automated RLS tests for each role in CI
Data residency	Data stored in India	Supabase and storage in Mumbai; Amplify hosting in Mumbai
Accessibility	Usable on low-end Android phones	Large touch targets, 16 px minimum text, no heavy client libraries

10. Running cost estimate

Running cost is about $30-50 a month for Bindushree alone and about $70-150 a month at 50 businesses and 1,000 users, roughly $1.50-3 per business.

Item	Launch: 1 tenant (USD/month)	50 tenants, ~1,000 users (USD/month)	Basis
Supabase	25	40-75	Pro $25 incl. $10 compute credit and 100,000 monthly active users; larger compute and storage at scale
Staging Supabase project	0	0-10	Free tier, or a second small Pro project
AWS Amplify Hosting	5	10-25	Build minutes and traffic
Email	0	0-20	Provider free tier, then a small plan
Sentry	0	0-26	Free tier, then a team plan
Domain and misc	2	2-5	
WhatsApp Business API (optional)	0	~3 + provider fee	About ₹0.145 per utility message in India; 1,600 messages a month
Total	**~30-50**	**~70-150**	Excludes GST and development time

The Tally connector adds no hosting cost; it runs on each business's PC. A Windows code-signing certificate (approx. $100-300 a year) is needed so the installer is trusted.

11. Build plan, risks and open decisions

Phase 1 takes about six weeks for one full-stack developer, ending with a two-week pilot at Bindushree before any second tenant is onboarded.

Phase 1 build sequence (approximate)

1. Week 1: repo, environments, auth, tenants and roles, RLS policies with tests.
2. Weeks 2-3: customers, equipment and chemicals masters, installed units, service events, PMD schedule and `pmd_status_v`, dashboard colours.

3. Week 4: service report form, PDF template, email and WhatsApp share, send log.
4. Week 5: public service-request form, Excel import of existing customers, daily reminder job, PWA offline drafts.
5. Week 6: pilot with Muttu and one technician; fix, then freeze Phase 1.

Phase 2 (signatures, assignment, request status) about 3 weeks; SaaS onboarding and Razorpay billing about 3 weeks; Tally connector and payments about 4-6 weeks.

Risks

Risk	Impact	Mitigation
A missed RLS policy leaks one tenant's data to another	High	RLS enabled on every table by migration check; automated cross-tenant tests in CI
Muttu's report format arrives late	Phase 1 slips	Build with a generic template; swap layout in `packages/pdf` when it arrives
Technicians resist a new app	Low adoption	Pilot with one technician, keep the visit form to one screen, pre-fill from the unit
PMD date rule misunderstood	Wrong reminders	Confirm BRD Q4 first; unit tests for each rule in `packages/core`
Tally versions and setups vary	Connector support load	Support TallyPrime only at first; test on 3 real customer setups before release
SMS OTP needs DLT registration	Customer login delayed	Magic link by email first; WhatsApp OTP later

Open decisions

- [ ] Product name and SaaS domain (separate from bindushreebiomedicals.com)
- [ ] Next PMD counted from due date or completion date (BRD Q4)
- [ ] Email provider: Resend or Amazon SES
- [ ] Customer login at launch: public form only, or magic-link portal
- [ ] SaaS pricing per business per month (sets which add-ons are worth automating)